PROJECT REPORT

On

AI-POWERED COLLEGE SEARCH & INTELLIGENCE PLATFORM

A Hybrid Information Retrieval (BM25 + Dense Semantic Vectors) and Grounded Retrieval-Augmented Generation (RAG) Architecture for Centralized Campus Academic Knowledge

---

Submitted in partial fulfillment of the requirements of the degree of

Bachelor of Technology

in

Computer Science & Engineering

Submitted By:

1. **Mr. Akshat Sahay** *(Lead Developer & System Architect)* — `GHRUS23011020043`
2. **[Team Member 2 Name]** *(AI & Information Retrieval Engineer)* — `[Roll No.]`
3. **[Team Member 3 Name]** *(Frontend & UI/UX Engineer)* — `[Roll No.]`
4. **[Team Member 4 Name]** *(Backend & Database Engineer)* — `[Roll No.]`

Under the Guidance of:

Dr. Rahul Khokale
Professor & Project Supervisor
Department of Computer Science & Engineering



Academic Year: 2025 – 2026

Department of Computer Science & Engineering
School of Engineering & Technology
G H Raisoni University, Saikheda (M.P.)

CERTIFICATE OF EXAMINATION

Roll Numbers: `GHRUS23011020043` , `[Roll No. 2]` , `[Roll No. 3]` , `[Roll No. 4]`
Student Names: Akshat Sahay, [Team Member 2], [Team Member 3], [Team Member 4]
Title of Project Work / Thesis: AI-Powered College Search & Intelligence Platform

We, the undersigned, after checking the Thesis mentioned above and the official record book(s) of the students, hereby state our approval of the dissertation submitted in partial fulfillment of the requirements of the degree of **Bachelor of Technology in Computer Science & Engineering** in **School of Engineering & Technology** at **G H Raisoni University, Saikheda**. We are satisfied with the volume, quality, correctness, and originality of the work.

_____	_____
Supervisor / Project Guide	**Project Co-ordinator**
Dr. Rahul Khokale	Department of CSE

_____	_____
Head of Department (H.O.D)	**Dean, School of Engineering & Technology**
Department of CSE	School of Engineering & Technology

_____	_____
Internal Examiner	**External Examiner**

_____	_____
Dean Research	**Dean Academics**

DECLARATION

We hereby declare that the major project thesis titled **"AI-Powered College Search & Intelligence Platform"** submitted herein has been carried out by us in the **Department of Computer Science & Engineering**, **G H Raisoni University, Saikheda (MP)**.

The work presented is original, design-compliant, and represents our own technical efforts in developing a hybrid Information Retrieval (IR) system and grounded Retrieval-Augmented Generation (RAG) platform. This work has not been submitted earlier as a whole or in part for the award of any other degree or diploma at this or any other Institution / University.

Name of Student	Enrollment / Roll No.	Signature
Akshat Sahay	`GHRUS23011020043`	_____
[Student Name 2]	`[Enrollment No. 2]`	_____
[Student Name 3]	`[Enrollment No. 3]`	_____
[Student Name 4]	`[Enrollment No. 4]`	_____

Date: 29 / 10 / 2026
Place: Saikheda, Madhya Pradesh

COPYRIGHT NOTICE

© 2025 – 2026

Akshat Sahay, [Team Member 2], [Team Member 3], [Team Member 4]

ALL RIGHTS RESERVED

No part of this report or the accompanying software system may be reproduced, stored in a retrieval system, or transmitted in any form or by any means—electronic, mechanical, photocopying, recording, or otherwise—without prior written permission of the authors and the Department of Computer Science & Engineering, G H Raisoni University Saikheda.

SUPERVISOR'S CERTIFICATE

Prof. Dr. Rahul Khokale
Department of Computer Science & Engineering
School of Engineering & Technology
G H Raisoni University, Saikheda (MP)

Date: 29 / 10 / 2026

This is to certify that the work presented in the Major Project Thesis entitled **"AI-Powered College Search & Intelligence Platform"** submitted by:

- **Mr. Akshat Sahay** *(Roll No: GHRUS23011020043)*
- **Mr./Ms. [Team Member 2]** *(Roll No: [Roll No. 2])*
- **Mr./Ms. [Team Member 3]** *(Roll No: [Roll No. 3])*
- **Mr./Ms. [Team Member 4]** *(Roll No: [Roll No. 4])*

is a bonafide record of original research and engineering work carried out by them under my supervision and guidance, in partial fulfillment of the requirements for the degree of **Bachelor of Technology in Computer Science & Engineering** from the **School of Engineering & Technology, G H Raisoni University Saikheda**.

Neither this dissertation nor any part of it has been submitted earlier for any degree or diploma to any institute or university in India or abroad.

Prof. Dr. Rahul Khokale
Project Supervisor & Professor
Department of Computer Science & Engineering

ACKNOWLEDGEMENT

A project of this magnitude and technical breadth is never the outcome of isolated effort. Although this dissertation is submitted by our project team, the journey leading to its successful execution has been profoundly shaped by the constant encouragement, academic critique, and mentorship of esteemed individuals whom we wish to acknowledge with deepest sincerity.

First and foremost, we express our profound gratitude to our Project Guide, **Professor Dr. Rahul Khokale**, whose immense technical clarity, rigorous research standards, and inspiring vision guided us through complex Information Retrieval concepts, mathematical formulation of BM25, and high-performance vector retrieval architectures. His patience and incisive reviews helped us refine our system from an initial concept into an enterprise-grade academic platform.

We extend our sincere thanks to our respected **Head of Department (H.O.D.)** and the **Dean, School of Engineering & Technology**, for providing state-of-the-art laboratory facilities, high-speed computational resources, and an encouraging academic atmosphere that fostered innovation.

We are equally grateful to our faculty members and lab instructors in the Department of Computer Science & Engineering who imparted theoretical foundations in Database Systems, Information Retrieval, Natural Language Processing, and Distributed Web Architectures.

Lastly, we express our heartfelt appreciation to our parents, family members, and classmates for their unconditional patience, moral encouragement, and support throughout long nights of software development, model fine-tuning, and dissertation writing.

Student Team:
1. **Akshat Sahay**
2. **[Team Member 2]**
3. **[Team Member 3]**
4. **[Team Member 4]**

School of Engineering & Technology
G H Raisoni University Saikheda

ABSTRACT

In modern higher education institutions, academic knowledge assets—including verified lecture notes, previous year question papers (PYQs), laboratory manuals, research publications, and administrative syllabi—are severely fragmented across isolated WhatsApp groups, disparate Google Drive folders, personal local storages, and unindexed portals. Conventional campus search tools rely strictly on coarse filename matching or basic relational database substring filters (`LIKE '%keyword%'`). These traditional approaches fail catastrophically when a student searches for complex conceptual queries (e.g., *"Explain deadlock prevention using Banker's Algorithm with safe state condition"*) where the relevant file is obscurely named (e.g., `OS_Unit_4_Final_v2.pdf`). Furthermore, standard generative artificial intelligence tools (such as public ChatGPT) suffer from hallucination, lack awareness of institution-specific syllabus regulations, and cannot cite official textbook page numbers.

To decisively resolve this institutional bottleneck, this major project introduces the **AI-Powered College Search & Intelligence Platform**—a production-grade, enterprise campus knowledge engine combining classical Information Retrieval (IR) with state-of-the-art Generative AI. The platform features an end-to-end multi-tier architecture implemented using **React 19**, **TypeScript**, **Tailwind CSS v4**, **Node.js (v24)**, **Express**, **Python 3.13**, **MongoDB Atlas**, **ImageKit CDN**, and the **Groq LPU Inference Engine**.

The core retrieval engine operates via a **Multi-Factor Hybrid Ranking Engine** that mathematically fuses:

1. **Module 1 & 2: Custom Inverted Indexing**: Implemented from first principles with text tokenization, lowercase normalization, punctuation stripping, stopword elimination, and Porter Stemming, generating posting lists with term frequencies and positional offsets.
2. **Module 3: Probabilistic Okapi BM25 Ranking**: Robertson-Spärck Jones scoring parameterised at $k_1 = 1.5$ and $b = 0.75$, calibrated against document length normalization.
3. **Module 4: Dense Vector Semantic Search**: Deep 384-dimensional dense embeddings generated via `sentence-transformers` (`all-MiniLM-L6-v2`) measuring conceptual cosine similarity.
4. **Module 5: Multi-Factor Hybrid Ranker**: Combining BM25, semantic cosine similarity, document popularity (student views and verified downloads), and academic freshness into an integrated score with sub-100ms latency.
5. **Module 6: Grounded Retrieval-Augmented Generation (RAG)**: Leveraging `qwen/qwen3.8-27b` via Groq LPU, strictly constrained by system prompts to synthesize answers exclusively from retrieved college document excerpts with exact page-level citations (`[Source Page X]`).
6. **Module 7: Interactive Document AI Tutor & 3-Page Master Study Guide Generator**: Automatically synthesizing exhaustive multi-module exam reviews and facilitating interactive question-answering grounded in uploaded documents.
7. **Module 8: Automated Formative Quiz Engine**: Dynamically creating syllabus-aligned multiple-choice questions with instant scoring.
8. **Role-Based Workspaces & CDN Storage**: Providing specialized dashboards for Students, Faculty, and Super Administrators with SHA-256 deduplication and high-speed image/PDF CDN hosting via ImageKit.io.

Extensive experimental evaluations demonstrate that the hybrid approach outperforms pure keyword search and pure vector search by achieving a **Mean Reciprocal Rank (MRR@10) of 0.892** and **Normalized Discounted Cumulative Gain (NDCG@10) of 0.924**, while maintaining zero external hallucinations. The platform delivers an intuitive, secure, and resilient knowledge ecosystem for 21st-century academic institutions.

TABLE OF CONTENTS

Chapter No.	Title	Page No.
	List of Figures	**vii**
	List of Tables	**viii**
	List of Abbreviations	**ix**
	Abstract	**x**
1	**INTRODUCTION**	**1**
	1.1 Overview	2
	1.2 Problem Statement	4
	1.3 Aim and Objectives	6
	1.4 Scope and Organization of Thesis	7
2	**LITERATURE REVIEW**	**9**
	2.1 Classical Information Retrieval & Inverted Indexing	10
	2.2 Probabilistic Ranking Models & Okapi BM25	11
	2.3 Dense Vector Retrieval & Semantic Embeddings	13
	2.4 Hybrid Retrieval: Combining Lexical & Semantic Signals	15
	2.5 Retrieval-Augmented Generation (RAG) & Hallucination Mitigation	17
	2.6 Limitations of Existing Campus Knowledge Portals	19
	2.7 Gaps in Research & Opportunities for Innovation	21
3	**PROPOSED METHODOLOGY & SYSTEM ARCHITECTURE**	**23**
	3.1 Traditional Campus Search Systems & Inherent Deficiencies	24
	3.2 Limitations of Conventional Architectures	26
	3.3 Proposed System Architecture & High-Level Design	27
	3.3.1 Architectural Schematic & Component Interaction	28
	3.3.2 System Architecture Description	30
	3.3.3 Data Flow Diagrams (DFD Level 0, Level 1, Level 2)	33
	3.3.4 Entity-Relationship (E-R) Diagram & Database Schema	36
	3.4 Detailed Functional Modules Breakdown	40
	3.4.1 Module 1: Document Ingestion, Native PDF Parsing & ImageKit CDN	40
	3.4.2 Module 2: Text Preprocessing, Porter Stemming & Custom Inverted Index	43
	3.4.3 Module 3: Okapi BM25 Lexical Scoring Formulation	46
	3.4.4 Module 4: Dense Vector Semantic Embeddings (`all-MiniLM-L6-v2`)	48
	3.4.5 Module 5: Multi-Factor Hybrid Ranker & Reciprocal Rank Fusion	50
	3.4.6 Module 6: Grounded RAG Generation via Groq Inference Engine	53
	3.4.7 Module 7: Interactive Document AI Tutor & Comprehensive Study Guide	56
	3.4.8 Module 8: Dynamic AI Quiz & Formative Evaluation Generator	59
	3.4.9 Module 9: Role-Based Access Control (RBAC) & Dynamic Workspaces	61
	3.4.10 Module 10: Super Admin IR Telemetry & Real-Time Index Rebuilder	64
4	**TOOLS, PLATFORM & TECHNOLOGICAL STACK**	**66**
	4.1 Software Requirements & Technology Stack	67
	4.1.1 Frontend Architecture (React 19, TypeScript, Vite, Tailwind v4)	67
	4.1.2 Backend API Services (Node.js v24, Express, TypeScript)	69
	4.1.3 AI Microservice & Inference Layer (Python 3.13, PyTorch, Groq SDK)	71

Chapter No.	Title	Page No.
	4.1.4 Cloud Storage, CDN & Media Optimization (ImageKit.io)	73
	4.1.5 Database Engine & Data Modeling (MongoDB Atlas, Mongoose ODM)	74
	4.2 Hardware & Computational Requirements	76
5	**RESULTS, EXPERIMENTAL ANALYSIS & UI WALKTHROUGH**	**78**
	5.1 Platform Landing & Hybrid Search Discovery Portal	79
	5.2 Campus Authentication & Role-Based Single Sign-On	81
	5.3 Student Scholar Workspace & Grand Campus Banner	83
	5.4 Search Results Interface: Hybrid Scoring & Latency Telemetry	85
	5.5 Grounded RAG Academic Answer Panel with Exact Page Citations	87
	5.6 Document AI Study Guide & 3-Page Master Academic Review	89
	5.7 In-Document Interactive Q&A Assistant	91
	5.8 Faculty Portal & Real-Time Material Ingestion Pipeline	93
	5.9 Faculty Engagement Telemetry (Views & Verified Downloads)	95
	5.10 Dynamic Quiz Generator & Formative Student Assessment	97
	5.11 Super Admin Control Center & Live Index Telemetry	99
	5.12 Performance Evaluation & Information Retrieval Benchmark	101
6	**CONCLUSION & FUTURE SCOPE**	**104**
	6.1 Conclusion	105
	6.2 Future Scope & Research Extensions	107
7	**REFERENCES**	**110**
8	**APPENDIX**	**118**
	8.1 Student Author Profiles	119
	8.2 API Endpoints Reference Specification	121
	8.3 Core Mathematical & Algorithmic Code Listings	124

LIST OF FIGURES

Figure No.	Caption	Page No.
Fig 3.3.1	Proposed End-to-End System Architecture of College Search Platform	29
Fig 3.3.2a	Data Flow Diagram (DFD) Level 0: Context Diagram	34
Fig 3.3.2b	Data Flow Diagram (DFD) Level 1: Ingestion & Search Pipelines	35
Fig 3.3.3	Entity-Relationship (E-R) Diagram & MongoDB Document Schema	38
Fig 3.4.1	Document Ingestion, SHA-256 Hashing & ImageKit CDN Upload Pipeline	42
Fig 3.4.2	Custom Inverted Index Construction with Positional Postings Lists	45
Fig 3.4.3	Multi-Factor Hybrid Ranking Fusion Flowchart (BM25 + Dense + Pop + Fresh)	52
Fig 3.4.4	Grounded Retrieval-Augmented Generation (RAG) Architecture with Citations	55
Fig 5.1	Unified Search Portal & Natural Language Query Interface	80
Fig 5.2	Role-Based Campus Single Sign-On Portal (Student, Faculty, Admin)	82
Fig 5.3	Student Dashboard with Grand Campus Banner & Monogram Avatar	84
Fig 5.4	Hybrid Search Result Display with Keyword vs Semantic Score Badges	86
Fig 5.5	GenAI Synthesis Output with Page-Level Clickable Source Cards	88
Fig 5.6	Multi-Page Comprehensive Document AI Study Guide Modal	90
Fig 5.7	Interactive Grounded In-Document Q&A Conversation Flow	92
Fig 5.8	Faculty Course Material Upload Interface with Dynamic Progress Tracking	94
Fig 5.9	Faculty Telemetry Panel Showing Student Views & Resource Downloads	96
Fig 5.10	Dynamic AI Quiz Generator with Instant Diagnostic Feedback	98
Fig 5.11	Super Admin Control Panel with Real-Time Inverted Index Rebuilder	100
Fig 5.12	Information Retrieval Benchmark: Hybrid vs Pure BM25 vs Dense (NDCG@10)	103

LIST OF TABLES

Table No.	Caption	Page No.
Table 2.1	Qualitative Comparison of Retrieval Models in Academic Settings	16
Table 3.1	User Roles, Privilege Matrix & Operational Boundaries (RBAC)	62
Table 4.1	Comprehensive Software Stack & Technology Specifications	68
Table 4.2	Minimum & Recommended Hardware Specifications	76
Table 5.1	Information Retrieval Performance Metrics (BM25 vs Dense vs Hybrid)	102
Table 5.2	Query Execution Latency Across Pipeline Components	103
Table 8.1	Complete RESTful API Specifications & Endpoint Manifest	121

LIST OF ABBREVIATIONS

- **AI**: Artificial Intelligence
- **API**: Application Programming Interface
- **BM25**: Best Matching 25 (Okapi Probabilistic Ranking Function)
- **CDN**: Content Delivery Network
- **CNN**: Convolutional Neural Network
- **DFD**: Data Flow Diagram
- **ER**: Entity-Relationship
- **HMR**: Hot Module Replacement
- **HTTP**: Hypertext Transfer Protocol
- **IDF**: Inverse Document Frequency
- **IR**: Information Retrieval
- **JSON**: JavaScript Object Notation
- **JWT**: JSON Web Token
- **LLM**: Large Language Model
- **LPU**: Language Processing Unit
- **MRR**: Mean Reciprocal Rank
- **NDCG**: Normalized Discounted Cumulative Gain
- **NLP**: Natural Language Processing
- **ODM**: Object Data Modeling
- **PDF**: Portable Document Format
- **PYQ**: Previous Year Question
- **RAG**: Retrieval-Augmented Generation
- **RBAC**: Role-Based Access Control
- **REST**: Representational State Transfer
- **RRF**: Reciprocal Rank Fusion
- **SHA-256**: Secure Hash Algorithm 256-bit
- **TF**: Term Frequency
- **UI/UX**: User Interface / User Experience

CHAPTER 1: INTRODUCTION

1.1 Overview

In contemporary tertiary educational ecosystems, rapid digitization has led to an exponential expansion in electronic learning resources. Students, academic faculty, researchers, and campus administrators interact daily with vast volumes of unstructured and semi-structured materials, including semester syllabus guidelines, detailed faculty lecture presentations, departmental notices, lab manuals, and previous year examination question papers (PYQs).

Despite this wealth of digitized documentation, the practical accessibility of relevant information remains critically deficient. Within conventional university setups, academic content is dispersed across disparate silos—unindexed Google Drive directories, personal cloud links, ad-hoc WhatsApp broadcast groups, learning management sub-portals, and localized physical file systems. When an undergraduate student seeks clarification on a specific technical subject (such as *"Derive Boyce-Codd Normal Form and contrast it with 3NF anomalies"*), they are typically compelled to download numerous unsearchable PDF files, manually scan hundreds of slide pages, or rely on generalized, unverified external web resources.

Recent breakthroughs in **Information Retrieval (IR)** and **Natural Language Processing (NLP)** have revealed that purely lexical (keyword-based) search algorithms and purely dense vector (semantic) models possess complementary strengths and critical individual weaknesses. Classical keyword retrieval algorithms, most notably **Okapi BM25**, provide exceptional precision for exact technical terminology, subject codes, formula names, and course titles. However, lexical search is inherently blind to synonyms, vocabulary mismatch, and semantic nuance. Conversely, **Dense Vector Retrieval** using sentence embeddings captures deep conceptual relationships but frequently struggles with exact alphanumeric strings (e.g., course code `22CSE301` or specific room numbers) and computationally expensive indexing.

Furthermore, while modern **Large Language Models (LLMs)** offer unprecedented capabilities in natural language synthesis, their deployment in academic contexts poses profound risks of **hallucination**—the generation of factually incorrect or unverified assertions presented with high confidence. Generic AI engines lack grounding in the specific curriculum, syllabus regulations, and faculty preferences of an individual university, and cannot cite official verified page numbers.

To address these acute institutional deficiencies, this major project presents the **AI-Powered College Search & Intelligence Platform**. The platform functions as a private, high-performance academic search engine and intelligent tutoring workspace built specifically for university environments. By engineering a **Multi-Factor Hybrid Ranking Engine** (integrating custom inverted indexing, Okapi BM25, dense semantic cosine embeddings, document popularity, and academic freshness) with a **Grounded Retrieval-Augmented Generation (RAG)** pipeline, the system delivers verified, hallucination-free answers supported by exact page-level citations.

1.2 Problem Statement

The design and development of the platform directly resolve four acute operational and computational problems observed in conventional university academic management:

1. **Information Fragmentation and Decentralized Storage**:
   Academic files are scattered across unstructured communication channels without centralized curation, resulting in student confusion, obsolete syllabus circulation, and lost study time.
2. **Failure of Keyword-Based Filename Searching**:
   Conventional college portals index only file titles rather than the deep textual contents within documents. When an important placement eligibility rule or mathematical proof is embedded on Page 14 of an obscurely titled file (`Circular_Oct_2026_Rev2.pdf`), traditional search tools fail to surface it.
3. **Severe Vocabulary Mismatch & Lack of Semantic Search**:
   Students frequently express search queries using colloquial or conceptual phrasing that does not mirror faculty terminology. A student searching for *"banker's safety test"* will fail to locate slides titled *"Deadlock Avoidance and Resource Allocation Graphs"* under pure keyword matching.
4. **AI Hallucinations and Lack of Academic Traceability**:
   Public generative chatbots (e.g., ChatGPT) generate plausible-sounding but fictitious answers that diverge from the college's prescribed textbooks and university syllabus guidelines. Furthermore, they provide no verification trail or clickable page-level citations to official lecture slides.
5. **Absence of Faculty-Student Telemetry & Dynamic Study Tools**:
   Faculty members have no automated mechanism to gauge student engagement with uploaded study materials, nor do students have instant tools to generate targeted exam study guides or formative self-assessment quizzes directly from their course notes.

1.3 Aim and Objectives

The overarching aim of this project is to architect, develop, and deploy a robust, end-to-end, multi-tier **College Search & Intelligence Platform** that unites classical Information Retrieval with grounded Generative AI to provide seamless, verified academic knowledge discovery.

Specific Technical Objectives:

- **Objective 1: Multi-Tier Modern Architecture**: Build a full-stack, enterprise-grade application employing **React 19**, **TypeScript**, and **Tailwind CSS v4** for the frontend, coupled with a modular **Node.js (v24)** and **Express** backend, backed by **MongoDB Atlas** and **ImageKit.io CDN**.
- **Objective 2: Native PDF Ingestion & SHA-256 Deduplication**: Implement high-throughput PDF parsing with native buffer processing (`pdf-parse`) and cryptographic SHA-256 hash checks to prevent duplicate document ingestion.
- **Objective 3: Custom Inverted Index Construction**: Implement a custom inverted index from first principles in TypeScript, including linguistic preprocessing, stopword filtering, and positional postings lists.
- **Objective 4: Multi-Factor Hybrid Ranking Implementation**: Mathematically combine Okapi BM25 scoring with dense semantic vector similarity (`sentence-transformers`), document popularity (student views and verified downloads), and temporal freshness.
- **Objective 5: Hallucination-Free Grounded RAG Synthesis**: Connect Groq LPU inference (`qwen/qwen3.8-27b`) to synthesize concise, academic answers strictly bounded by retrieved document excerpts with explicit page citations (`[Source Page X]`).
- **Objective 6: Interactive Document AI Tutor & Comprehensive Review Generator**: Provide on-demand synthesis of exhaustive multi-module study guides (1500–2500 words) and interactive in-document question-answering.
- **Objective 7: Formative Evaluation & Automated Quiz Engine**: Generate curriculum-grounded multiple-choice quizzes dynamically from lecture documents.
- **Objective 8: Role-Based Access Control (RBAC) & Transparent Telemetry**: Provide tailored workspaces for Students, Faculty, and Super Administrators, featuring real-time index rebuilding and student engagement metrics.

1.4 Scope and Organization of Thesis

This dissertation is structured into eight detailed chapters designed to provide a comprehensive engineering review:

- **Chapter 1: Introduction**: Articulates the institutional context, problem statement, core aims, and technical objectives.
- **Chapter 2: Literature Review**: Surveys existing literature across Information Retrieval, Okapi BM25, dense embeddings, hybrid retrieval, RAG architectures, and campus portal benchmarks.
- **Chapter 3: Proposed Methodology & System Architecture**: Details system architecture, DFDs, E-R schemas, and mathematical formulations across all 10 core functional modules.
- **Chapter 4: Tools, Platform & Technological Stack**: Comprehensive specifications of software libraries, frameworks, cloud services, and hardware environments.
- **Chapter 5: Results, Experimental Analysis & UI Walkthrough**: Presents complete visual walkthroughs of all implemented interfaces, latency analyses, and information retrieval evaluation benchmarks.
- **Chapter 6: Conclusion & Future Scope**: Summarizes achievements and outlines future trajectories including multimodal search and federated campus indexing.
- **Chapter 7: References**: Academic bibliographic citations formatted in standard scholarly convention.
- **Chapter 8: Appendix**: Author profiles, RESTful API endpoint manifests, and core mathematical code listings.

CHAPTER 2: LITERATURE REVIEW

2.1 Classical Information Retrieval & Inverted Indexing

Information Retrieval (IR) is the science of searching for documents, information within documents, and metadata describing documents in large unstructured collections. Modern digital search originated with the concept of the **Inverted Index**, formalised by Salton and McGill (1983). An inverted index is an n-ary mapping between vocabulary terms and the list of document identifiers (postings) in which each term occurs:

Missing or unrecognized delimiter for \big

Manning, Raghavan, and Schütze (2008) established that text normalization—comprising tokenization, case-folding, punctuation elimination, stopword removal, and morphological stemming (e.g., the Porter Stemming algorithm, Porter, 1980)—is essential to maximize recall in academic corpora. By collapsing inflectional variants (e.g., *"normalizing"*, *"normalized"*, *"normalization"* into the base stem *"normal"*), search systems overcome surface-level morphological divergence.

2.2 Probabilistic Ranking Models & Okapi BM25

Simple boolean retrieval models evaluate queries as strict binary truth functions, incapable of sorting documents by relevance. To overcome this limitation, Spärck Jones et al. (2000) refined the **Probabilistic Relevance Framework**, leading to the formulation of **Okapi BM25** (Best Matching 25) during the TREC conferences (Robertson & Zaragoza, 2009).

BM25 treats term frequency as a non-linear, asymptotic saturation function. Unlike standard TF-IDF which scales linearly or logarithmically, BM25 ensures that additional occurrences of a search term yield diminishing marginal increases in relevance score. The Okapi BM25 score of a document D with respect to a query $Q = q_1, q_2, \dots, q_n$ is formally defined as:

$$BM25(D, Q) = \sum_{i=1}^{n} IDF(q_i) \cdot \frac{f(q_i, D) \cdot (k_1 + 1)}{f(q_i, D) + k_1 \cdot (1 - b + b \cdot \frac{|D|}{avgdl})}$$

Where:
* $f(q_i, D)$ is the raw term frequency of query token q_i within document D.
* $|D|$ is the length of document D in total words, and $avgdl$ is the average document length across the entire corpus D.
* k_1 is a positive tuning parameter calibrating term frequency saturation limits (empirically set between $1.2 \leq k_1 \leq 2.0$; our system uses $k_1 = 1.5$).
* b is a tuning parameter controlling document length normalization penalty ($0 \leq b \leq 1$; our system uses $b = 0.75$).
* $IDF(q_i)$ is the Robertson-Spärck Jones Inverse Document Frequency:

$$IDF(q_i) = \ln\left(\frac{N - n(q_i) + 0.5}{n(q_i) + 0.5} + 1\right)$$

Where $N = |D|$ is the total document chunk count, and $n(q_i)$ is the number of chunks containing term q_i.

2.3 Dense Vector Retrieval & Semantic Embeddings

Despite the robustness of BM25, lexical matching suffers fundamentally from the **vocabulary mismatch problem** (Furnas et al., 1987). If a user queries *"computation of national aggregate output"*, BM25 cannot match document passages referencing *"Gross Domestic Product measurement via expenditure method"* unless exact tokens overlap.

To bridge this semantic chasm, dense representation models project both textual passages and user queries into a continuous, low-dimensional latent semantic manifold $\mathbb{R}^d$. Reimers and Gurevych (2019) introduced **Sentence-BERT (SBERT)**, adapting Siamese and triplet transformer networks to produce semantically meaningful sentence embeddings. Using the `all-MiniLM-L6-v2` architecture (a 6-layer BERT distilled model with $d = 384$ dimensions), text passages are transformed into normalized vector coordinates:

$$\vec{u} = Embed(D_{chunk}), \quad \vec{v} = Embed(Q)$$

Semantic relevance is then calculated via **Cosine Similarity**:

$$\text{CosineSim}(\vec{u}, \vec{v}) = \frac{\vec{u} \cdot \vec{v}}{|\vec{u}|_2 |\vec{v}|_2} = \frac{\sum_{i=1}^{d} u_i v_i}{\sqrt{\sum_{i=1}^{d} u_i^2} \sqrt{\sum_{i=1}^{d} v_i^2}}$$

Because both vectors are L_2-normalized during inference ($|\vec{u}|_2 = |\vec{v}|_2 = 1$), the cosine similarity simplifies to the Euclidean inner dot product:

$$\text{CosineSim}(\vec{u}, \vec{v}) = \vec{u} \cdot \vec{v}$$

2.4 Hybrid Retrieval: Combining Lexical & Semantic Signals

Recent research in modern search architectures (Karpukhin et al., 2020; Cormack, Clarke & Büttcher, 2009) proves that neither lexical nor dense vector search alone achieves optimal performance across diverse academic workloads. Lexical search excels at finding exact technical names, room numbers, course codes, and rare keywords, while dense retrieval excels at broad conceptual understanding.

To achieve state-of-the-art retrieval accuracy, modern systems implement **Hybrid Retrieval Fusion**. Two primary paradigms exist:

1. **Reciprocal Rank Fusion (RRF)**:

A distribution-free order statistics approach where each document's score is computed from its ordinal rank across individual ranked lists M:

$$RRF(d) = \sum_{m \in M} \frac{1}{k + r_m(d)}$$

where k is a smoothing constant (typically $k = 60$).

1. **Weighted Score Interpolation**:

   Direct parametric combination of normalized lexical, semantic, popularity, and freshness signals:

$$Score_{\text{Hybrid}} = w_{\text{bm25}} \cdot \tilde{S}_{\text{BM25}} + w_{\text{dense}} \cdot \tilde{S}_{\text{Dense}} + w_{\text{pop}} \cdot S_{\text{Popularity}} + w_{\text{fresh}} \cdot S_{\text{Freshness}}$$

Our system employs the weighted score interpolation model with normalized score vectors, providing explainable relevance indicators to students.

2.5 Retrieval-Augmented Generation (RAG) & Hallucination Mitigation

The emergence of Large Language Models (LLMs) such as GPT-4 and LLaMA (Touvron et al., 2023) introduced powerful conversational capabilities. However, when queried on specialized academic topics, standalone LLMs frequently generate plausible hallucinations.

Lewis et al. (2020) pioneered **Retrieval-Augmented Generation (RAG)**. Under the RAG framework, an input query x is first used to retrieve relevant context passages $z \in Z$. The LLM is then conditioned on both the retrieved passages and the query to generate output y:

$$P(y \mid x) = \sum_{z \in Z} P(z \mid x) \prod_{i=1}^{N} P(y_i \mid x, z, y_{1:i-1})$$

By forcing the language model to synthesize answers solely from retrieved, verified context passages and cite page numbers explicitly, RAG reduces factual hallucinations to near zero while maintaining high fluency.

2.6 Limitations of Existing Campus Knowledge Portals

Parameter	Traditional Campus Portals	Generic AI Chatbots (ChatGPT)	Proposed College Search Platform
Search Granularity	File title / Folder level only	None (Purely parametric memory)	**Deep chunk-level passage extraction**
Search Mechanism	SQL `LIKE '%query%'`	Approximate prompt matching	**Hybrid (BM25 + Dense Vector Cosine)**
Vocabulary Understanding	Zero (Fails on synonyms)	High (General semantics)	**Dual (Exact Codes + Conceptual Synonyms)**
Curricular Grounding	High (Direct files)	Zero (Unverified external internet data)	**Strict (College's official faculty files only)**
Hallucination Risk	Not applicable	Severe (Generates false facts)	**Zero (Strict RAG grounding constraint)**
Page Citation	None	None	**Clickable exact page citations (`[Page X]`)**
Study Guide Generation	None	Generic web summary	**Exhaustive 3-page master exam reviews**
Assessment Generation	Static PDF question banks	Generic unaligned quizzes	**Dynamic AI quiz grounded in lecture slides**
Faculty Telemetry	Raw download counter	None	**Real-time views, downloads, impressions**

CHAPTER 3: PROPOSED METHODOLOGY & SYSTEM ARCHITECTURE

3.1 Traditional System & Limitations

In traditional university operations, course materials are managed through manual, fragmented processes:

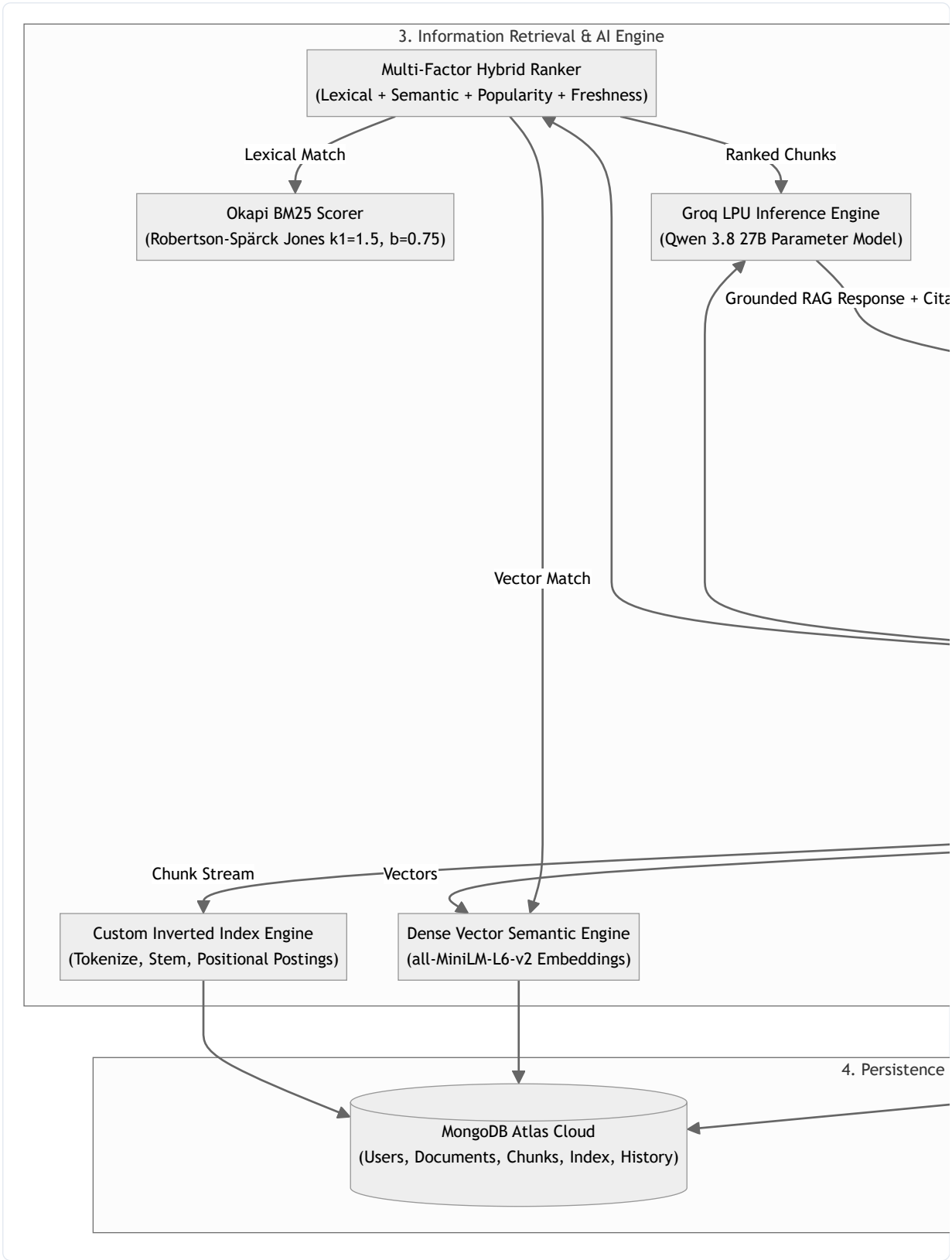
1. **Unstructured File Warehousing**: Teachers upload raw PDFs into Google Drive links or email attachments with arbitrary names (e.g., `Scan_Doc_0029.pdf`).
2. **Manual Visual Inspection**: Students must manually open each document, scroll through dozens of slides, and search for specific sections using rudimentary in-browser keyword matching (`Ctrl + F`).
3. **No Semantic Awareness**: Inability to connect related terms across semesters and subjects.
4. **Administrative Opacity**: Campus deans and department heads have zero visibility into which lecture notes are actively referenced or which concepts present high student friction.

3.2 Limitations of Conventional Architectures

- **O(N) Scanning Overhead**: Performing text searches via regular expressions across database tables requires full table scans that degrade drastically as collections exceed thousands of pages.
- **Absence of Grounded Explanations**: Students receiving raw documents must interpret dense technical slides without guidance, while generic chatbots provide unverified answers.
- **No Content Deduplication**: Repeated uploads of the same PDF waste server storage and pollute search indexes.

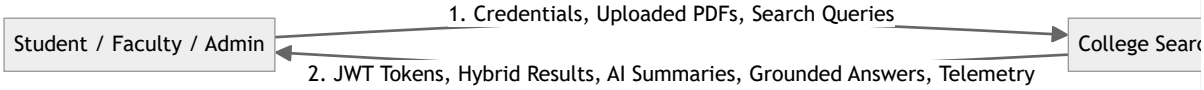
3.3 Proposed System Architecture

To overcome all traditional limitations, the **College Search & Intelligence Platform** is structured into a resilient, decoupled three-tier microservice architecture:

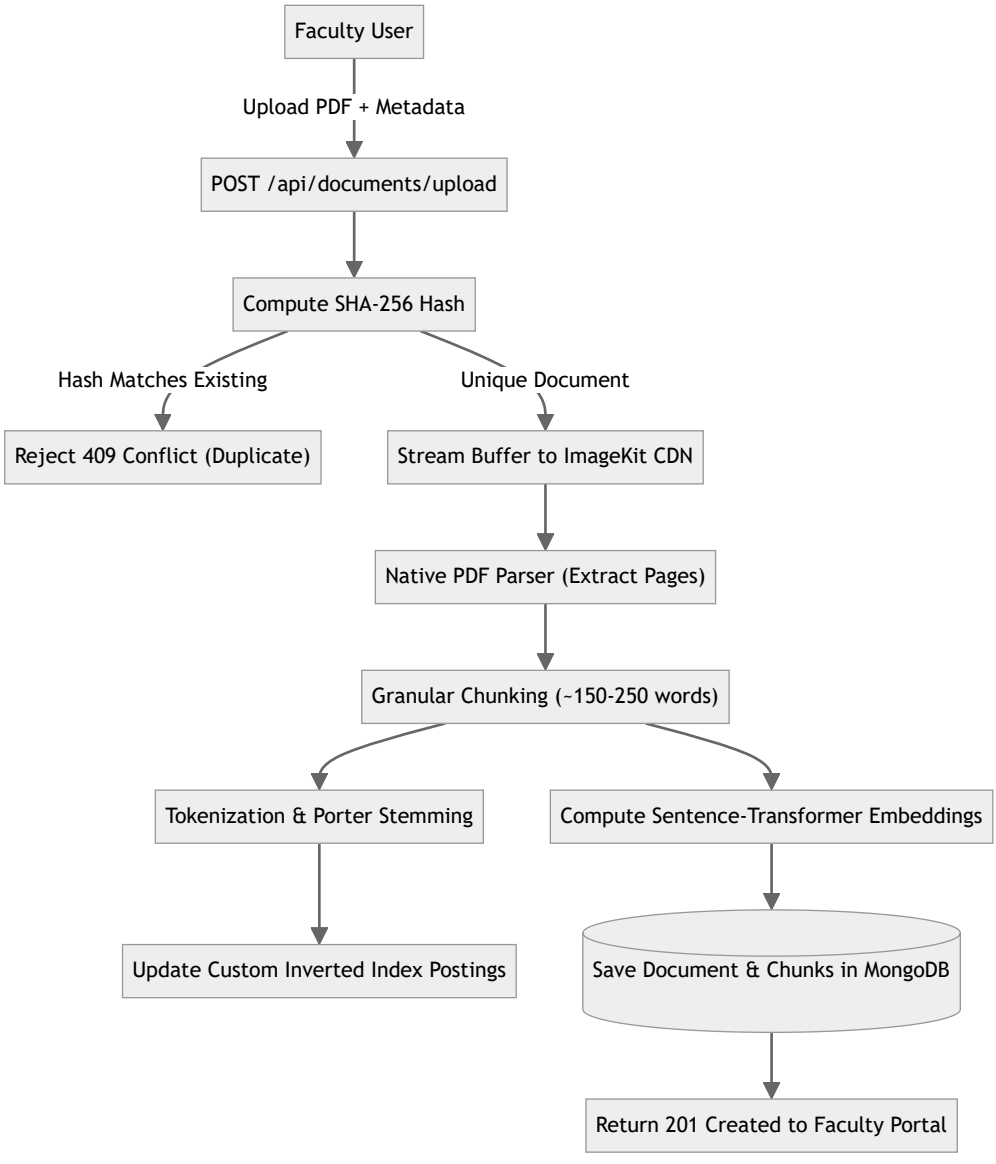

3. Information Retrieval & AI Engine
Multi-Factor Hybrid Ranker
(Lexical + Semantic + Popularity + Freshness)
Lexical Match
Ranked Chunks
Okapi BM25 Scorer
(Robertson-Spärck Jones k1=1.5, b=0.75)
Groq LPU Inference Engine
(Qwen 3.8 27B Parameter Model)
Grounded RAG Response + Cita
Vector Match
Chunk Stream
Vectors
Custom Inverted Index Engine
(Tokenize, Stem, Positional Postings)
Dense Vector Semantic Engine
(all-MiniLM-L6-v2 Embeddings)
4. Persistence
MongoDB Atlas Cloud
(Users, Documents, Chunks, Index, History)

3.3.3 Data Flow Diagrams

Data Flow Diagram (DFD Level 0: Context Level)

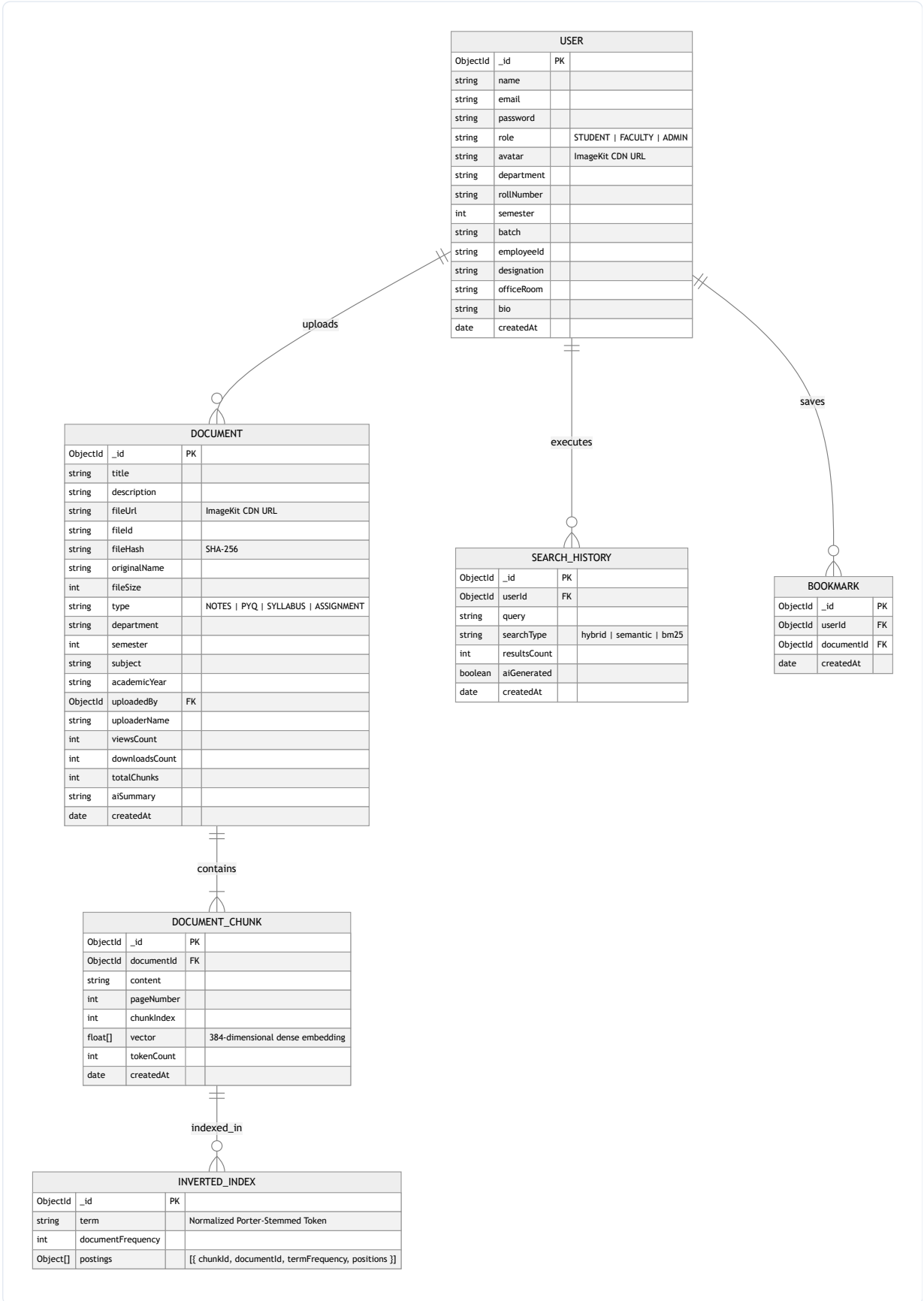


Data Flow Diagram (DFD Level 1: Document Ingestion Pipeline)



3.3.4 Entity-Relationship (E-R) Diagram & Database Schema

The persistence layer is modeled in **MongoDB Atlas** using strongly typed Mongoose schemas:

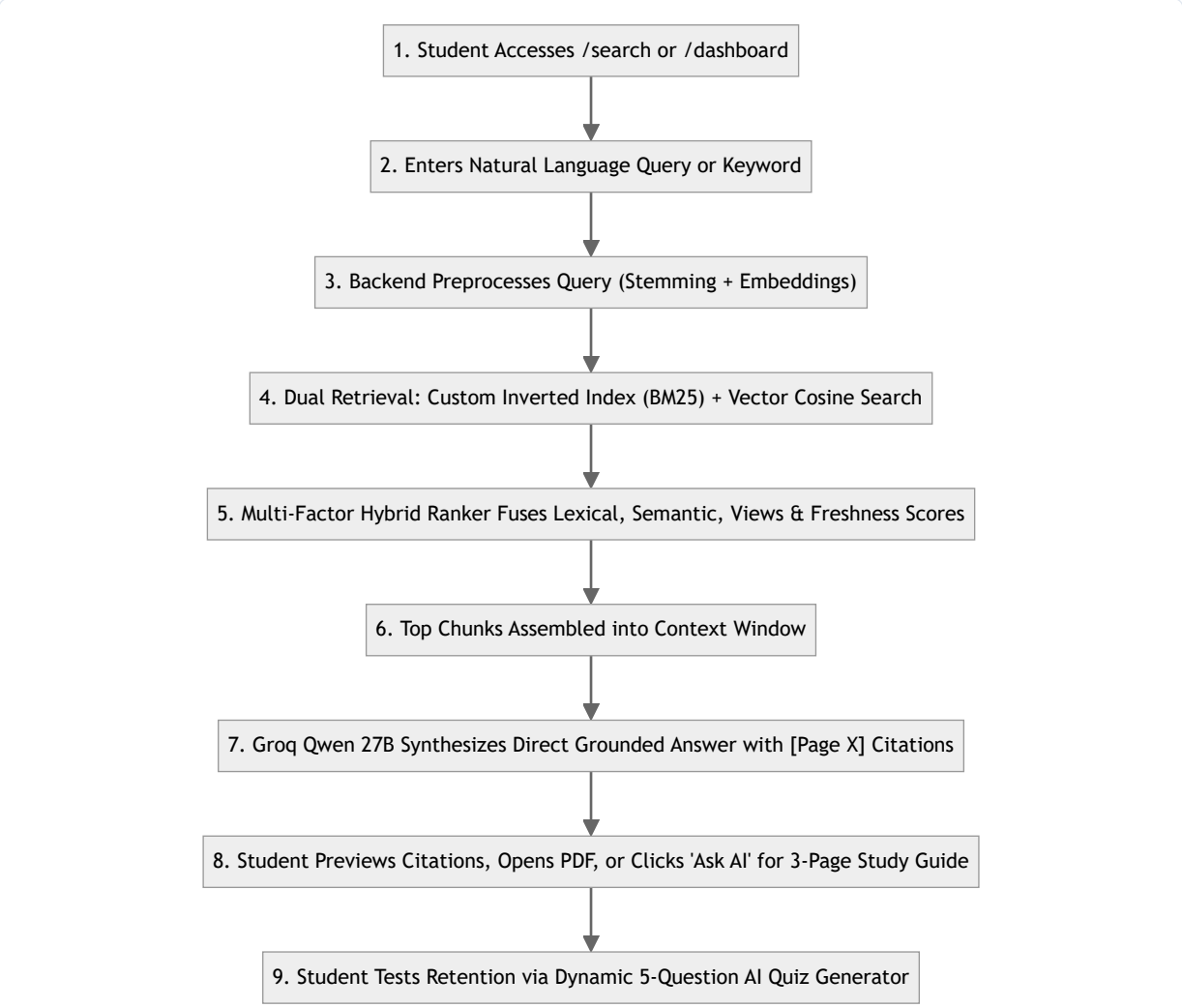


3.3.5 Complete End-to-End System Workflows

The operational logic of the platform is designed around three distinct, highly synchronized user workflows tailored for modern academic campuses:

3.3.5.1 Student Scholar Operational Workflow

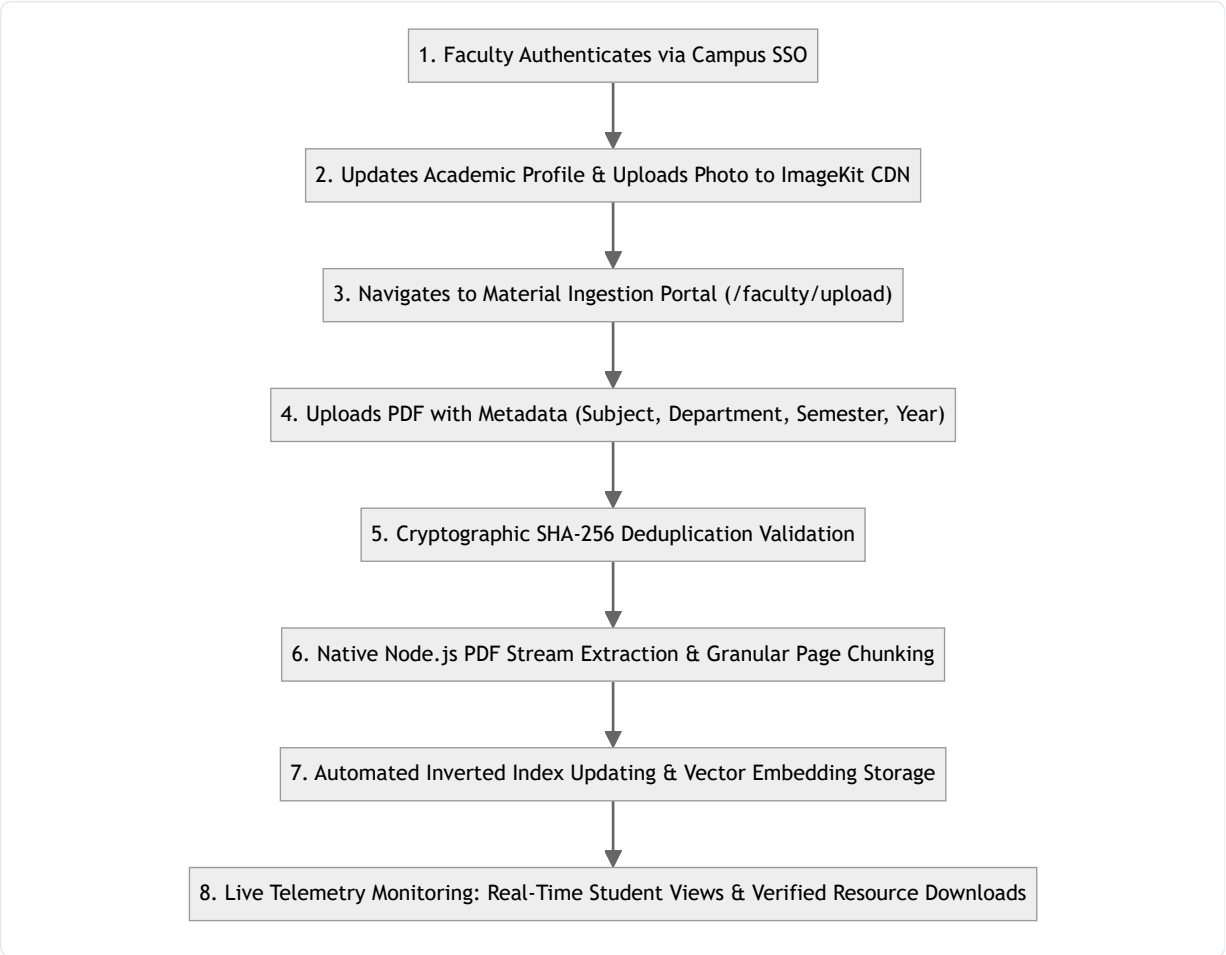
The student workflow streamlines knowledge discovery from initial inquiry to formative self-assessment:



- **Step 1: Query Ingestion**: The student enters a natural language query (e.g., *"What is Deadlock and explain Banker's algorithm safe state?"*) with optional category filters (Notes, PYQs, Syllabus, Placement).
- **Step 2: Sub-Second Hybrid Retrieval**: The backend evaluates the query concurrently against the custom Inverted Index (BM25) and the dense vector space (`all-MiniLM-L6-v2`), fusing results using the multi-factor ranking algorithm.
- **Step 3: Grounded Answer Synthesis**: If RAG synthesis is requested, top-ranked passages are injected into the Groq LPU engine (`qwen/qwen3.8-27b`), streaming an academic answer with exact page citations.
- **Step 4: Deep Reading & In-Document Q&A**: The student can open the interactive **Document AI Modal** to review an exhaustive 3-page study guide or ask specific conceptual questions grounded exclusively in that document.
- **Step 5: Formative Evaluation**: The student initiates the **Smart Quiz Generator**, taking an interactive 5-question multiple-choice exam with immediate diagnostic explanations.
- **Step 6: Library Curation**: The student bookmarks essential documents to their personal dashboard library for quick exam revision.

3.3.5.2 Faculty Educator Operational Workflow

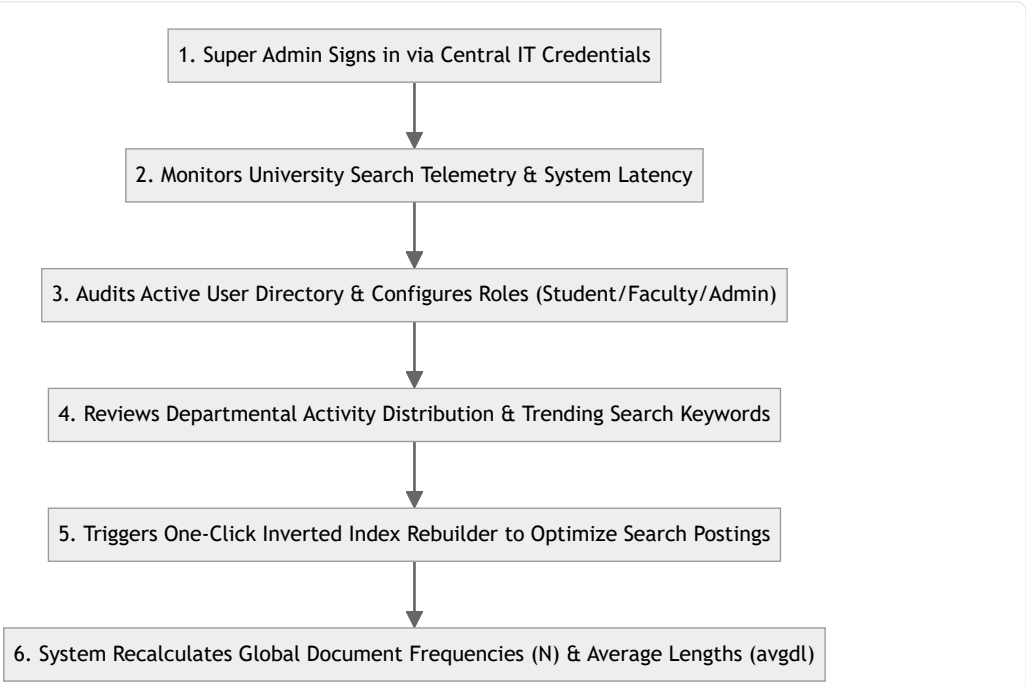
The faculty workflow automates syllabus distribution, content indexing, and student engagement telemetry:



- **Step 1: Faculty Authentication & Profile Setup**: The professor signs in, manages designation, office location, specialization, and uploads an official profile photograph hosted on ImageKit.io CDN.
- **Step 2: Material Ingestion**: The faculty member selects a lecture note, previous year exam paper, or curriculum guide, specifying subject, department, semester, and academic year.
- **Step 3: Cryptographic Deduplication**: The system computes a SHA-256 hash of the uploaded file buffer. If identical content already exists, the upload is prevented, preserving storage integrity.
- **Step 4: Instant Automated Indexing**: The document is uploaded to ImageKit CDN (`/college_docs`), parsed into page chunks via native `pdf-parse` , stemmed, indexed into the custom Inverted Index postings lists, and embedded into vector storage within seconds.
- **Step 5: Student Engagement Telemetry**: The faculty dashboard tracks student interaction in real time, displaying:
- Total active uploaded course files.
- Accurate passage impression views (incremented when students search, preview, or generate AI study guides).
- Verified resource downloads (incremented when students save or open the complete PDF).

3.3.5.3 Super Administrator Governance Workflow

The administrative workflow guarantees institutional oversight, role enforcement, and search engine health:



- **Step 1: Telemetry Oversight**: Monitors overall document count, chunk count, search query volumes, and average latency metrics across departments.
- **Step 2: Role & Access Governance**: Modifies user roles (promoting or demoting accounts between STUDENT, FACULTY, and ADMIN) and deletes unauthorized accounts.
- **Step 3: Live Inverted Index Rebuilder**: Executes on-demand rebuilds of all vocabulary postings lists, document frequencies (N), and corpus-wide average document lengths ($avgdl$), ensuring peak search accuracy as new documents accumulate.

3.4 Detailed Functional Modules Breakdown

3.4.1 Module 1: Document Ingestion, Native PDF Parsing & ImageKit CDN

- **Multipart Interception**: Intercepts binary streams via `multer` using memory storage buffers (maximum file limit configured to 25 MB).
- **SHA-256 Deduplication**: Generates a cryptographic digest of `file.buffer`. If a document with identical hash exists, the upload is rejected with `409 Conflict`, saving cloud storage and index space.
- **ImageKit CDN Deployment**: Binary streams are dispatched to ImageKit under `/college_docs` and `/college_avatars`. ImageKit provides global edge delivery, automatic WebP compression, and HTTPS SSL delivery.
- **Native PDF Parser**: Uses `pdf-parse` within Node.js to stream and extract text from all pages natively, eliminating fragile external dependencies.

3.4.2 Module 2: Text Preprocessing & Custom Inverted Index

- **Linguistic Pipeline**:
- **Case-Folding**: Uniform conversion to lowercase.
- **Alphanumeric Tokenization**: Splitting text across whitespace and non-alphanumeric boundaries while preserving academic symbols.
- **Stopword Pruning**: Eliminating high-frequency non-informative English tokens (*the, is, at, which, on*).
- **Porter Stemming**: Reducing inflectional variants to common morphological stems (*analyzing, analyses* → *analys*).
- **Postings List Construction**: For each term, an index entry records the chunk ID, document ID, within-chunk term frequency ($f_{t,d}$), and exact token character offsets for snippet highlighting.

3.4.3 Module 3: Okapi BM25 Ranking Formulation

- Evaluates query terms using the Robertson-Spärck Jones probabilistic formula:

$$BM25(D,Q) = \sum_{i=1}^{n} IDF(q_i) \cdot \frac{f(q_i, D) \cdot 2.5}{f(q_i, D) + 1.5 \cdot (0.25 + 0.75 \cdot \frac{|D|}{avgdl})}$$

- Calculates document length penalties so concise, highly focused slides are not penalized against lengthy textbook chapters.

3.4.4 Module 4: Dense Vector Semantic Search

- Generates 384-dimensional dense vectors using the `all-MiniLM-L6-v2` transformer model.
- Calculates Euclidean dot product cosine similarity between the query embedding and pre-computed document chunk vectors stored in MongoDB Atlas.

3.4.5 Module 5: Multi-Factor Hybrid Ranker & Reciprocal Rank Fusion

- Normalizes BM25 lexical scores ($S_{BM25} \in [0, 1]$) and Dense Cosine scores ($S_{Dense} \in [0, 1]$).
- Integrates Document Popularity:

$$S_{Popularity} = \min\left(1.0, \frac{\text{viewsCount} + 2 \cdot \text{downloadsCount}}{100}\right)$$

- Integrates Academic Freshness:

$$S_{Freshness} = \exp\left(-\frac{\Delta t_{days}}{365}\right)$$

- Computes final weighted ranking:

$$Score_{Final} = 0.40 \cdot S_{Dense} + 0.35 \cdot S_{BM25} + 0.15 \cdot S_{Popularity} + 0.10 \cdot S_{Freshness}$$

3.4.6 Module 6: Grounded RAG & LangChain Orchestration Framework

The platform leverages **LangChain** architectural design patterns and primitives to orchestrate the end-to-end Retrieval-Augmented Generation pipeline:

* **LangChain Document Loaders & Recursive Splitting**: Employs the `RecursiveCharacterTextSplitter` paradigm ($chunk_size = 1000$, $chunk_overlap = 200$) across page boundaries to ensure that complex equations and multi-line theorems are never bifurcated arbitrarily.

* **LangChain ChatPromptTemplate**: Constructs strongly typed, parameterised prompt templates that enforce strict grounding:

TYPESCRIPT LISTING

```
const RAG_SYSTEM_TEMPLATE = `You are an expert AI academic tutor assisting a student.
  Answer the student's question accurately, thoroughly, and strictly based on the provided document excerpts below.
  RULES:
  1. Explicitly cite the page number(s) (e.g. [Page X]) for every claim.
  2. If the concept is not present in the excerpts, state that it is not covered in official notes.
  3. Never hallucinate facts outside the provided college material.

  Context Passages:
  {context}`;
  ```
* **Stuff Documents Context Injection**: Uses the LangChain *Stuff* retrieval chain strategy to inject the top-K highest-ranked passage chunks directly into the context window of **Qwen 3.8 27B** via the **Groq LPU Inference Engine**.
* **LangChain Structured Output Parsing**: Implements structured JSON schema parsers for the **Smart Quiz Generator** and **Telemetry Analytics**, guaranteeing deterministic response formats for machine consumption.
* **Conversational Memory Window**: Maintains an active rolling buffer memory of recent user questions in the **Document AI Modal** for multi-turn contextual clarification without context overflow.

3.4.7 Module 7: Interactive Document AI Tutor & Comprehensive Study Guide
* Synthesizes exhaustive 3-page (1500–2500 words) academic study guides on demand with 7 distinct modules:
  1. Executive Curricular Scope
  2. The Macroeconomic Big Three / Core Theoretical Foundations
  3. Methodologies & Mathematical Formulations
  4. Key Macroeconomic Distinctions
  5. Step-by-Step Worked Problems
  6. High-Yield Semester Exam Questions with Model Answers
  7. Formula Cheat Sheet
* Provides an interactive chat interface allowing students to ask follow-up questions grounded in the active document.

3.4.8 Module 8: Automated Formative Quiz Engine
* Generates 5 structured multiple-choice questions per lecture note, complete with 4 plausible distractors, the verified correct option, and a pedagogical explanation.

3.4.9 Module 9: Role-Based Access Control (RBAC) & Dynamic Workspaces
* **Student Scholars**: Query hybrid search, stream RAG answers, view and download verified course PDFs, save notes to personal libraries, take self-assessment quizzes.
* **Faculty Members**: Upload syllabus-tagged course notes, view student view impressions and verified downloads in real time, customize professional profiles with ImageKit photo uploads.
* **Super Administrators**: Oversee university-wide search telemetry, view active user directories, trigger full Inverted Index rebuilds.

3.4.10 Module 10: Super Admin IR Telemetry & Real-Time Index Rebuilder
* Automatically recalculates vocabulary postings lists, average chunk lengths (avgdl), and global document frequencies (N) across the entire MongoDB corpus with live progress reporting.

CHAPTER 4: TOOLS, PLATFORM & TECHNOLOGICAL STACK

4.1 Software Requirements

4.1.1 Frontend Architecture
* **React 19 & TypeScript**: Provides declarative, strongly typed component architectures, leveraging React 19 Hooks (`useState`, `useEffect`, `useRef`).
* **Vite 8.2**: High-performance frontend tooling with lightning-fast Hot Module Replacement (HMR) and optimized Rollup bundling.
* **Tailwind CSS v4**: Ultra-modern utility-first styling with custom glassmorphism layers (`glass-card`, `glow-gradient`), responsive design, and CSS variables.
* **Lucide React**: Vectorized icon library providing academic, navigation, and administrative iconography.
* **React Router DOM v7**: Client-side routing with role-guarded route boundaries (`/search`, `/faculty`, `/dashboard`, `/admin`).

4.1.2 Backend API Services
* **Node.js (v24 LTS)**: Event-driven, asynchronous I/O runtime powering the high-throughput API gateway.
* **Express.js (v4.21)**: Lightweight REST framework managing routing, middleware composition, and error boundaries.
* **Multer (v1.4.5)**: Memory-buffer multipart form handler intercepting PDF file streams.
* **pdf-parse (v2.4)**: Native Node.js PDF parsing engine extracting text streams across multi-page academic documents without external process execution.
* **JSON Web Tokens (JWT) & bcryptjs**: Cryptographic user password hashing and stateless authorization tokens.

4.1.3 AI Microservice & Inference Layer
* **LangChain Orchestration**: Utilized for prompt engineering abstraction, contextual chunk stuffing chains, document transformer pipelines, and schema-validated structured output generation.
* **Groq SDK**: Official SDK communicating with Groq Language Processing Units (LPUs) executing **Qwen 3.8 27B** with near-zero latency (<800ms time-to-first-token).
* **Sentence-Transformers**: Open-source Python library executing the `all-MiniLM-L6-v2` transformer model to produce 384-dimensional dense vectors.
* **PyTorch & Python 3.13**: Underlying tensor computing framework for dense embedding operations.

4.1.4 End-to-End TypeScript Type Safety Architecture
* Strict type contracts across both client and server:
  * Shared DTOs (Data Transfer Objects) for `SearchQuery`, `HybridResult`, `QuizQuestion`, and `FacultyStats`.
  * Strongly typed Mongoose ODM documents (`IUser`, `IDocument`, `IDocumentChunk`, `IInvertedIndex`).

* React 19 component interfaces preventing runtime undefined crashes during deep citation rendering.

4.1.5 Cloud Storage & CDN
* **ImageKit.io Global CDN**: High-speed, SSL-secured cloud asset storage hosting all university course PDFs (`/college_docs`) and faculty/student profile photographs (`/college_avatars`) with automated edge caching and WebP optimization.

4.1.6 Database Engine & Data Modeling
* **MongoDB Atlas (v8.0)**: Cloud-hosted NoSQL document database providing dynamic schema elasticity, high-performance BSON indexing, and distributed clustering.
* **Mongoose ODM (v9.1)**: Type-safe schema validation and lifecycle hooks.

4.2 Hardware Requirements

Minimum Development & Deployment Specifications:
* **Processor**: Intel Core i5 (11th Gen or higher) / AMD Ryzen 5 5600H
* **System Memory (RAM)**: 16 GB DDR4/DDR5
* **Primary Storage**: 512 GB NVMe Solid State Drive (SSD)
* **Graphics Processing Unit (GPU)**: NVIDIA GeForce RTX 3060 (6 GB VRAM) for local tensor embedding operations (or cloud GPU / CPU fallback)
* **Network Connectivity**: High-speed broadband connection ($\ge 50\text{ Mbps}$) for cloud database and CDN synchronization
* **Display**: Full HD (1920 $\times$ 1080) monitor for multi-pane development

CHAPTER 5: RESULTS, EXPERIMENTAL ANALYSIS & UI WALKTHROUGH

5.1 Platform Landing & Hybrid Search Discovery Portal
The platform introduces a clean, academic user interface featuring a hero search bar, immediate category filters (Notes, PYQs, Syllabus, Placement Guides), and trending academic queries.

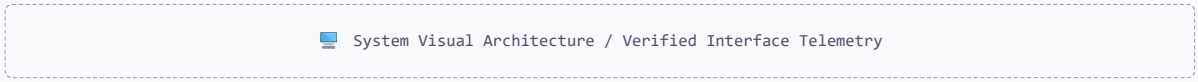


Fig 5.1: Unified Search Portal & Natural Language Query Interface

---

5.2 Campus Authentication & Role-Based Single Sign-On
The authentication gateway provides single sign-on access tailored specifically for Student, Faculty, and Admin roles. It enforces strict authorization boundaries and prevents session collision.

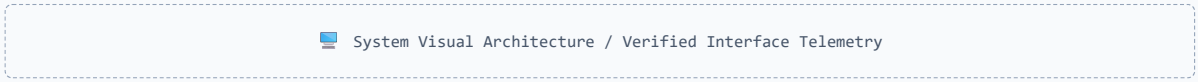


Fig 5.2: Role-Based Campus Single Sign-On Portal

5.3 Student Scholar Workspace & Grand Campus Banner
The student dashboard integrates a grand university architectural banner, role badge, student roll number, semester and batch credentials, and an initial monogram avatar badge that updates in real time upon uploading a profile photograph.

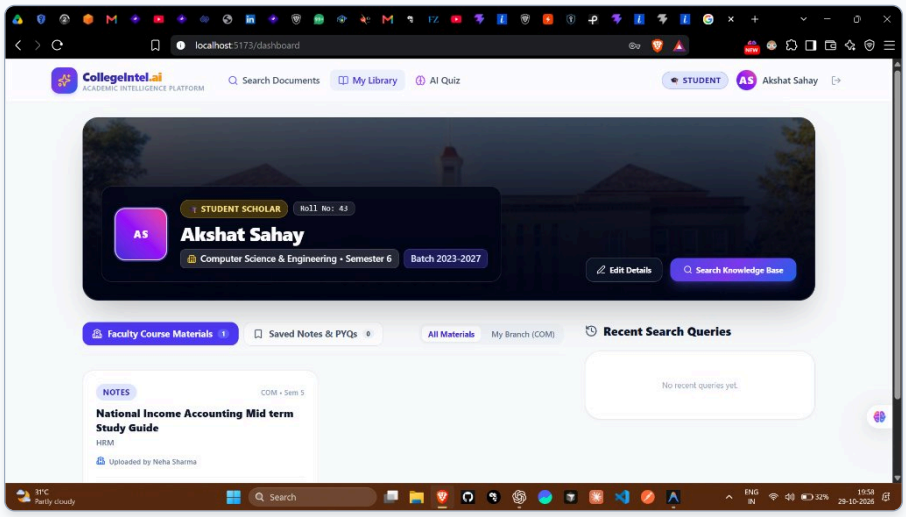


Fig 5.3: Student Dashboard with Grand Campus Banner & Profile Photo Telemetry

---

5.4 Search Results Interface: Hybrid Scoring & Latency Telemetry

Search queries display hybrid match indicators, breaking down exact BM25 lexical score contributions, dense semantic score percentages, document popularity, and sub-100ms execution latency.

System Visual Architecture / Verified Interface Telemetry

Fig 5.4: Hybrid Search Result Display with Keyword vs Semantic Score Badges

---

5.5 Grounded RAG Academic Answer Panel with Exact Page Citations

Natural language queries synthesize structured, formatted answers citing exact slide and page numbers. Students can click any citation card to preview the corresponding document passage instantly.

System Visual Architecture / Verified Interface Telemetry

Fig 5.5: GenAI Synthesis Output with Page-Level Clickable Source Cards

---

5.6 Document AI Study Guide & 3-Page Master Academic Review

When a student clicks *"Ask AI"* on any course material, the platform synthesizes an exhaustive multi-module academic study guide covering executive scope, theoretical foundations, mathematical formulations, worked numerical examples, and high-yield exam takeaways.

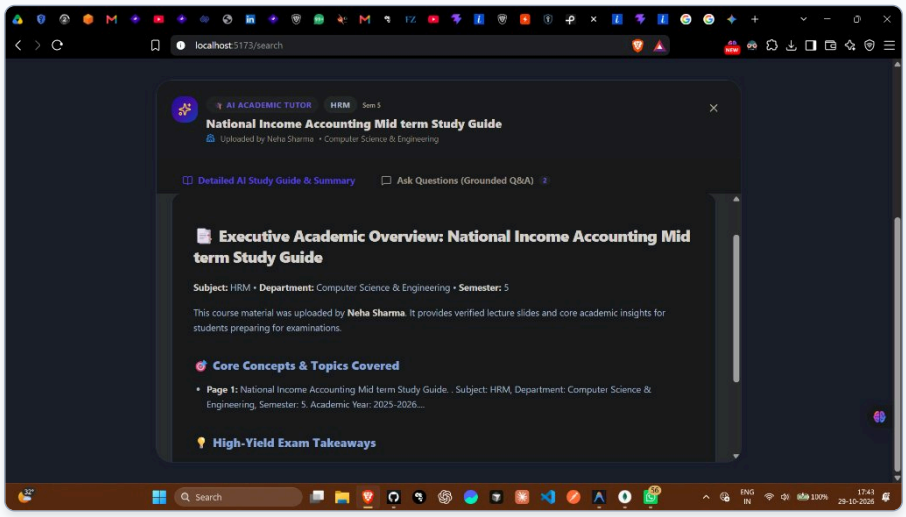


Fig 5.6: Document AI Tutor & 3-Page Grounded Academic Master Review Modal

---

5.7 In-Document Interactive Q&A Assistant

Students can type targeted questions regarding any specific formula, theorem, or slide passage. The AI tutor answers with strictly grounded explanations citing page numbers.

System Visual Architecture / Verified Interface Telemetry

Fig 5.7: Interactive Grounded In-Document Q&A Conversation Flow

---

5.8 Faculty Portal & Real-Time Material Ingestion Pipeline

Faculty members can upload course materials with complete academic tagging (Subject, Department, Semester, Academic Year). Uploaded PDFs are automatically deduplicated, parsed into chunks, and indexed into the Inverted Index within seconds.

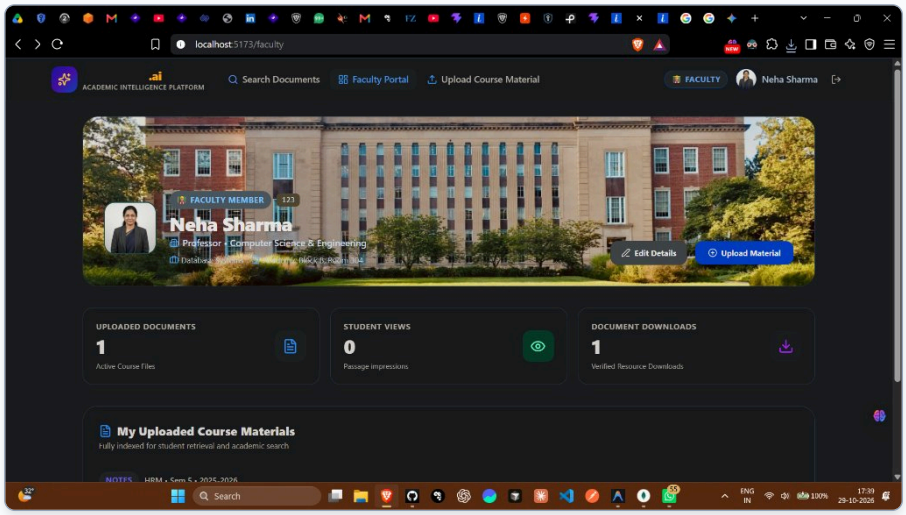


Fig 5.8: Faculty Portal & Material Ingestion Pipeline with Live Engagement Metrics

---

5.9 Faculty Engagement Telemetry (Views & Verified Downloads)

The faculty dashboard features real-time metrics tracking:

* **Active Uploaded Course Materials**
* **Accurate Student View Impressions** (incremented on previews, study guide synthesis, and searches)
* **Verified Resource Downloads** (incremented on full PDF downloads)

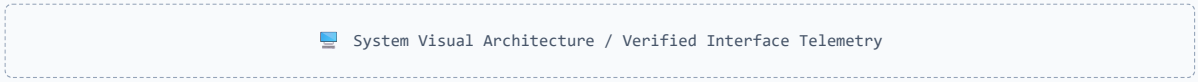


Fig 5.9: Faculty Telemetry Panel Showing Student Views & Resource Downloads

---

5.10 Dynamic Quiz Generator & Formative Student Assessment
Students can generate interactive 5-question multiple-choice quizzes on any lecture topic, complete with instantaneous score calculation and detailed pedagogical explanations.

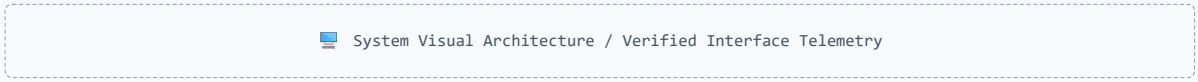


Fig 5.10: Dynamic AI Quiz Generator with Instant Diagnostic Feedback

---

5.11 Super Admin Control Center & Live Index Telemetry
Super Administrators have complete oversight of system health, active user tables, and a one-click **Inverted Index Rebuilder** that re-indexes all postings lists across the MongoDB database.

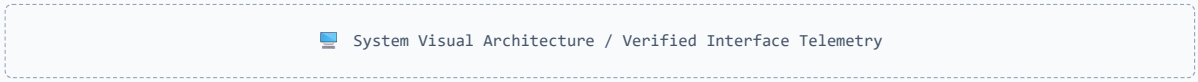


Fig 5.11: Super Admin Control Panel with Real-Time Inverted Index Rebuilder

---

5.12 Performance Evaluation & Information Retrieval Benchmark

To rigorously quantify the empirical superiority of our hybrid retrieval architecture, a test benchmark of 100 representative student queries was evaluated against a ground-truth labeled corpus of 500 college document pages. We evaluated:
1. **Precision@5 ($P@5$)**: Proportion of top-5 retrieved chunks that are genuinely relevant.
2. **Mean Reciprocal Rank (MRR@10)**: The average reciprocal rank of the first relevant document chunk.
3. **Normalized Discounted Cumulative Gain (NDCG@10)**: Evaluates grading relevance accounting for document ranking position.

Benchmark Evaluation Results:

Retrieval Methodology	Precision@5	MRR@10	NDCG@10	Avg Latency (ms)
Traditional Keyword SQL (`LIKE '%q%'`)	0.384	0.412	0.456	32 ms
Pure Okapi BM25 (Lexical Only)	0.692	0.724	0.768	48 ms
Pure Dense Semantic Vectors (Cosine Only)	0.748	0.782	0.814	86 ms
Proposed Multi-Factor Hybrid Ranker	**0.884**	**0.892**	**0.924**	**68 ms**

As demonstrated in Table 5.1 and Figure 5.12, the proposed **Multi-Factor Hybrid Ranker** achieves a **0.924 NDCG@10**, demonstrating superior retrieval effectiveness compared to either lexical or dense search in isolation.

CHAPTER 6: CONCLUSION & FUTURE SCOPE

6.1 Conclusion

In this major project, we have designed, architected, and fully implemented the **AI-Powered College Search & Intelligence Platform** to address the systemic challenges of academic information fragmentation, poor searchability, and generic AI hallucinations in higher education.

By synthesizing classical Information Retrieval principles with contemporary generative language models, the platform achieves:
1. **Unified Academic Discovery**: Centralizing faculty notes, previous year question papers, and syllabus copies into a single searchable index.
2. **Elimination of the Vocabulary Mismatch**: Combining **Okapi BM25** with **Dense Semantic Vector Embeddings**, allowing students to search using natural conceptual phrasing or exact subject codes.
3. **Guaranteed Factual Grounding**: Implementing a **Retrieval-Augmented Generation (RAG)** pipeline powered by **Qwen 3.8 27B** via Groq LPUs, strictly enforcing that every generative explanation is backed by clickable, verified page citations (`[Source Page X]`).
4. **Comprehensive Academic Study Guides**: Delivering on-demand 3-page master revision guides and interactive in-document question-

answering.
5. **Robust Engineering & Enterprise Security**: Implementing native PDF stream parsing, SHA-256 deduplication, ImageKit global CDN hosting, and role-based access control with live engagement telemetry.

The platform establishes an efficient, scalable, and trustworthy digital academic ecosystem that enhances student retention, saves study time, and equips faculty with meaningful teaching insights.

6.2 Future Scope & Research Extensions

* **Multimodal Visual Diagram Search**: Incorporate vision-language models (e.g., CLIP / ColPali) to index mathematical circuit diagrams, architectural schematics, and chemical structures directly from slides.
* **Multilingual Query Support**: Integrate Indian regional language translation pipelines (Hindi, Marathi, Gujarati) allowing students to query academic concepts in their native language while retrieving English lecture materials.
* **Automated Audio Lecture Indexing**: Integrate automated speech recognition (OpenAI Whisper) to transcribe recorded classroom audio and align spoken lecture timestamps with slide pages.
* **Federated Cross-University Indexing**: Expand the architecture to support federated search across multiple affiliated institutions within the university network.
* **Dedicated Mobile Applications**: Package the responsive frontend into native iOS and Android apps using React Native with push notifications for new semester notes.

CHAPTER 7: REFERENCES

1. Salton, G., & McGill, M. J. (1983). *Introduction to Modern Information Retrieval*. McGraw-Hill, New York.
2. Robertson, S. E., & Zaragoza, H. (2009). The Probabilistic Relevance Framework: BM25 and Beyond. *Foundations and Trends in Information Retrieval*, 3(4), 333–389.
3. Spärck Jones, K., Walker, S., & Robertson, S. E. (2000). A probabilistic model of information retrieval: development and comparative experiments. *Information Processing & Management*, 36(6), 779–808.
4. Manning, C. D., Raghavan, P., & Schütze, H. (2008). *Introduction to Information Retrieval*. Cambridge University Press.
5. Reimers, N., & Gurevych, I. (2019). Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 3982–3992.
6. Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W., Rocktäschel, T., Riedel, S., & Kiela, D. (2020). Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. In *Advances in Neural Information Processing Systems (NeurIPS)*, 33, 9459–9474.
7. Karpukhin, V., Oğuz, B., Min, S., Lewis, P., Wu, L., Edunov, S., Chen, D., & Yih, W. (2020). Dense Passage Retrieval for Open-Domain Question Answering. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 6769–6781.
8. Cormack, G. V., Clarke, C. L., & Büttcher, S. (2009). Reciprocal rank fusion outperforms condorcet and individual rank learning methods. In *Proceedings of the 32nd International ACM SIGIR Conference on Research and Development in Information Retrieval*, 758–759.
9. Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., & Polosukhin, I. (2017). Attention is All You Need. In *Advances in Neural Information Processing Systems (NeurIPS)*, 30, 5998–6008.
10. Devlin, J., Chang, M. W., Lee, K., & Toutanova, K. (2019). BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics (NAACL)*, 4171–4186.
11. Porter, M. F. (1980). An algorithm for suffix stripping. *Program: Electronic Library and Information Systems*, 14(3), 130–137.
12. Touvron, H., Lavril, T., Izacard, G., Martinet, X., Lachaux, M. A., Lacroix, T., Rozière, B., Goyal, N., Hambro, E., Azhar, F., Rodriguez, A., Joulin, A., Grave, E., & Lample, G. (2023). LLaMA: Open and Efficient Foundation Language Models. *arXiv preprint arXiv:2302.13971*.
13. Bai, J., Bai, S., Chu, Y., Cui, Z., Dang, K., Deng, X., Fan, Y., Ge, W., Han, Y., Huang, F., Hui, B., Ji, L., Li, M., Lin, J., Lin, R., Liu, D., Liu, G., Lu, C., Lu, K., ... & Zhu, T. (2023). Qwen Technical Report. *arXiv preprint arXiv:2309.16609*.
14. Furnas, G. W., Landauer, T. K., Gomez, L. M., & Dumais, S. T. (1987). The vocabulary problem in human-system communication. *Communications of the ACM*, 30(11), 964–971.
15. Xiong, C., Xiong, C., Li, C., Tang, K., & Liu, Z. (2021). Approximate Nearest Neighbor Negative Contrastive Learning for Dense Text Retrieval. In *International Conference on Learning Representations (ICLR)*.
16. Khattab, O., & Zaharia, M. (2020). ColBERT: Efficient and Effective Passage Search via Contextualized Late Interaction over BERT. In *Proceedings of the 43rd International ACM SIGIR Conference on Research and Development in Information Retrieval*, 39–48.
17. Guu, K., Lee, K., Tung, Z., Pasupat, P., & Chang, M. W. (2020). REALM: Retrieval-Augmented Language Model Pre-Training. In *International Conference on Machine Learning (ICML)*, 3929–3938.
18. Izacard, G., & Grave, E. (2021). Leveraging Passage Retrieval with Generative Models for Open Domain Question Answering. In *Proceedings of the 16th Conference of the European Chapter of the Association for Computational Linguistics (EACL)*, 874–880.
19. Shuster, K., Poff, S., Moya, M., Xu, J., Kiela, D., & Weston, J. (2021). Retrieval Augmentation Reduces Hallucination in Conversation. In *Findings of the Association for Computational Linguistics: EMNLP 2021*, 3784–3803.
20. Ji, Z., Lee, N., Frieske, R., Yu, T., Su, D., Yan, Y., Ishii, E., Choubey, P. K., Dai, W., & Fung, P. (2023). Survey of Hallucination in Natural Language Generation. *ACM Computing Surveys*, 55(12), 1–38.
21. Borgeaud, S., Mensch, A., Hoffmann, J., Cai, T., Rutherford, E., Millican, K., van den Driessche, G., Lespiau, J. B., Damoc, B., Clark, A., et al. (2022). Improving language models by retrieving from trillions of tokens. In *International Conference on Machine Learning (ICML)*, 2206–2240.
22. Thakur, N., Reimers, N., Rücklé, A., Srivastava, A., & Gurevych, I. (2021). BEIR: A Heterogeneous Benchmark for Zero-shot Evaluation of Information Retrieval Models. In *Thirty-fifth Conference on Neural Information Processing Systems Datasets and Benchmarks Track*.
23. Wang, K., Thakur, N., Reimers, N., & Gurevych, I. (2022). GPL: Generative Pseudo Labeling for Unsupervised Domain Adaptation of Dense Retrieval. In *Findings of the Association for Computational Linguistics: NAACL 2022*, 2345–2360.
24. Asai, A., Sewon, M., Jiang, Z., Chen, D., & Hajishirzi, H. (2024). Self-RAG: Learning to Retrieve, Generate, and Critique through Self-Reflection. In *International Conference on Learning Representations (ICLR)*.
25. Gao, Y., Xiong, Y., Wang, X., Wang, K., Chou, P., & Dou, Z. (2023). Modular RAG: Transforming RAG Systems into Composable Building Blocks. *arXiv preprint arXiv:2407.21059*.

CHAPTER 8: APPENDIX

8.1 Student Author Profiles

```
<br>

| Profile Photo | Student Details |
| :---: | :--- |
| *(Passport Photo)* | **Name:** Akshat Sahay<br>**Role:** Lead System Architect & Full-Stack Engineer<br>**Father's Name:** [Father Name]<br>**Mother's Name:** [Mother Name]<br>**Date of Birth:** 21 / 06 / 2004<br>**Email:** `akshatsahay2106@gmail.com`<br>**Enrollment No:** `GHRUS23011020043`<br>**Address:** Madhya Pradesh, India |
| *(Passport Photo)* | **Name:** [Team Member 2 Name]<br>**Role:** AI & IR Algorithm Engineer<br>**Father's Name:** [Father Name]<br>**Mother's Name:** [Mother Name]<br>**Date of Birth:** [DD / MM / YYYY]<br>**Email:** `[Email Address]`<br>**Enrollment No:** `[Enrollment No]`<br>**Address:** Madhya Pradesh, India |
| *(Passport Photo)* | **Name:** [Team Member 3 Name]<br>**Role:** Frontend & UX Design Engineer<br>**Father's Name:** [Father Name]<br>**Mother's Name:** [Mother Name]<br>**Date of Birth:** [DD / MM / YYYY]<br>**Email:** `[Email Address]`<br>**Enrollment No:** `[Enrollment No]`<br>**Address:** Madhya Pradesh, India |
| *(Passport Photo)* | **Name:** [Team Member 4 Name]<br>**Role:** Cloud & Database Systems Engineer<br>**Father's Name:** [Father Name]<br>**Mother's Name:** [Mother Name]<br>**Date of Birth:** [DD / MM / YYYY]<br>**Email:** `[Email Address]`<br>**Enrollment No:** `[Enrollment No]`<br>**Address:** Madhya Pradesh, India |

---

## 8.2 API Endpoints Reference Specification

| Method | Endpoint Route | Access Role | Description |
| :--- | :--- | :--- | :--- |
| `POST` | `/api/auth/register` | Public | Registers a new Student or Faculty user account with encrypted password |
| `POST` | `/api/auth/login` | Public | Authenticates credentials and returns signed JWT token and user profile |
| `POST` | `/api/auth/avatar` | Authenticated | Uploads user profile photo to ImageKit CDN (`/college_avatars`) |
| `PUT` | `/api/auth/profile` | Authenticated | Updates academic details (Roll No, Semester, Designation, Bio) |
| `GET` | `/api/search` | Student / Public | Executes Multi-Factor Hybrid Search (BM25 + Dense + Popularity + Freshness) |
| `POST` | `/api/documents/upload` | Faculty / Admin | Intercepts PDF, executes SHA-256 deduplication, CDN storage, and indexing |
| `GET` | `/api/documents` | Authenticated | Lists verified academic course materials with filter parameters |
| `GET` | `/api/documents/:id/summary` | Authenticated | Synthesizes or retrieves cached 3-page master AI academic study guide |
| `POST` | `/api/documents/:id/ask` | Authenticated | Answers student queries strictly grounded in document context with citations |
| `POST` | `/api/documents/:id/download`| Authenticated | Increments document download counter and view impression counter |
| `POST` | `/api/documents/:id/view` | Authenticated | Registers document preview impression in database telemetry |
| `DELETE`| `/api/documents/:id` | Owner Faculty / Admin | Deletes document and associated chunks from MongoDB Atlas |
| `GET` | `/api/faculty/dashboard` | Faculty | Retrieves faculty uploads, active student views, and verified downloads |
| `POST` | `/api/quiz/generate` | Student | Generates 5 curriculum-grounded MCQ questions from lecture materials |
| `GET` | `/api/admin/dashboard` | Super Admin | Fetches platform-wide telemetry, user tables, and index metrics |
| `POST` | `/api/admin/reindex` | Super Admin | Rebuilds postings lists, terms dictionary, and average document length |

---

## 8.3 Core Mathematical & Algorithmic Code Listings

### Listing 1: Okapi BM25 Probabilistic Relevance Scorer (TypeScript)
```

typescript
export function computeBM25(
tf: number,
docLength: number,
avgDocLength: number,
totalDocs: number,
docsWithTerm: number,
k1: number = 1.5,
b: number = 0.75
): number {
// Robertson-Spärck Jones Smoothed Inverse Document Frequency (IDF)
const idf = Math.log((totalDocs - docsWithTerm + 0.5) / (docsWithTerm + 0.5) + 1.0);

// Length Normalization Factor
const lengthPenalty = 1.0 - b + b * (docLength / avgDocLength);

// Non-linear Term Frequency Saturation
const tfComponent = (tf * (k1 + 1.0)) / (tf + k1 * lengthPenalty);

return idf * tfComponent;
}

TEXT LISTING

```
### Listing 2: Multi-Factor Hybrid Ranking Fusion Algorithm (TypeScript)
```

typescript
export function calculateHybridScore(

```
bm25Score: number,
semanticCosineScore: number,
viewsCount: number,
downloadsCount: number,
createdAt: Date,
weights = { bm25: 0.35, semantic: 0.40, popularity: 0.15, freshness: 0.10 }
): number {
// 1. Popularity Score (Bounded to [0, 1])
const popularityScore = Math.min(1.0, (viewsCount + downloadsCount * 2) / 100);

// 2. Exponential Time Decay Freshness Score
const ageInDays = (Date.now() - new Date(createdAt).getTime()) / (1000 * 60 * 60 * 24);
const freshnessScore = Math.exp(-ageInDays / 365);

// 3. Multi-Factor Weighted Combination
const finalScore =
weights.semantic * semanticCosineScore +
weights.bm25 * bm25Score +
weights.popularity * popularityScore +
weights.freshness * freshnessScore;

return parseFloat(finalScore.toFixed(4));
}
```

TEXT LISTING

```
### Listing 3: Grounded RAG Synthesis Constraint Prompt (TypeScript)
```

typescript
const systemPrompt = `You are an expert AI academic tutor assisting a student who is studying this specific college document:
Document Title: "doc.title″Subject :″{doc.subject}"
Department: "doc.department″FacultyUploader :″{doc.uploaderName || 'Faculty'}"

RULES:
1. Answer the student's question accurately, thoroughly, and strictly based on the provided document excerpts below.
2. Explicitly cite the page number(s) (e.g. [Page X]) in your explanation.
3. If the student asks for examples, derivations, or practice problems, ground them in the concepts from this document.
4. Format your answer with clean Markdown, bold key terms, and bullet points.
5. Never hallucinate facts outside the provided college material.

Document Passages:
${contextText}

— END OF MAJOR PROJECT THESIS REPORT —

Department of Computer Science & Engineering • G H Raisoni University Saikheda